

TECHNICAL BENCHMARK REPORT

# Evaluating Large Language Models for RTL Generation

A reproducible comparison across VerilogEval v2, RTLLM 2.0, ProtocolLLM, and RTL-OPT

**Prepared by:** SiliconCraft RTL Benchmarking Project  
**Report date:** 14 June 2026  
**Models evaluated:** Five public LLM baselines  
**Compute:** Lanta GPU cluster, OpenAI-compatible vLLM endpoints  
**Status:** Completed baseline comparison

**Bottom line** Qwen3.6 27B is the strongest overall model for functional RTL generation. DeepSeek-Coder-V2-Lite-Instruct leads the RTL-OPT equivalence test, showing the best behavior-preservation rate, but it rarely produces a smaller equivalent circuit.

## Executive Summary

We built a reusable benchmark harness that can swap models through an OpenAI-compatible API while keeping prompts, sampling settings, extraction logic, and evaluation tools consistent. Five models were evaluated under matched conditions.

- 1 Qwen3.6 27B achieved the best VerilogEval v2 functional pass@1 (61.54%) and pass@5 task recovery (77.56%).
- 1 Qwen3.6 27B and Qwen3.6 35B-A3B tied for the best RTLLM 2.0 functional result (60.00%).
- 1 DeepSeek Coder achieved the best RTL-OPT equivalence pass rate (72.50%), ahead of Qwen3.6 27B (62.50%).
- 1 Qwen2.5 Coder produced the strongest VerilogEval syntax rate, but that did not translate into functional correctness.
- 1 ProtocolLLM results are lint-only and must not be interpreted as protocol functional correctness.

## Recommendation

Use Qwen3.6 27B as the current default baseline for general RTL generation. Keep DeepSeek Coder as a complementary candidate for optimization workflows where behavior preservation is the first gate. Do not select a model based on syntax or lint scores alone.

# 1. What Was Evaluated

The benchmark suite covers three distinct questions. These measurements are deliberately reported separately because they do not prove the same capability.

| Benchmark      | Question answered                                                     | Trusted metric                    | Scale          |
|----------------|-----------------------------------------------------------------------|-----------------------------------|----------------|
| VerilogEval v2 | Can the model generate functionally correct RTL from a specification? | Icarus simulation pass@1 / pass@5 | 156 tasks      |
| RTLLM 2.0      | Can the model solve a second public RTL generation task set?          | Icarus functional pass@1          | 50 tasks       |
| ProtocolLLM    | Does generated protocol RTL parse and lint?                           | Verilator lint pass only          | 9 public tasks |
| RTL-OPT        | Can the model rewrite RTL while preserving behavior?                  | Yosys equivalence pass            | 40 tasks       |

## Models

- 1 Qwen3.6 27B
- 1 Qwen3.6 35B-A3B
- 1 Qwen3 Coder
- 1 Qwen2.5 Coder
- 1 DeepSeek Coder

## Controlled Settings

All comparisons use the same benchmark version, task set, neutral prompt template, extraction logic, timeout policy, and simulator/evaluation flow. Sampling settings were matched by benchmark condition.

| Condition                     | Samples/task | Temperature | Top-p | Max tokens |
|-------------------------------|--------------|-------------|-------|------------|
| VerilogEval pass@1            | 1            | 0.2         | 0.95  | 2,048      |
| VerilogEval pass@5            | 5            | 0.6         | 0.95  | 2,048      |
| RTLLM / ProtocolLLM / RTL-OPT | 1            | 0.2         | 0.95  | 4,096      |

**Interpretation rule** A lint pass is not a functional pass. A synthesis pass is not proof of behavior preservation. For RTL-OPT, only equivalence-passing outputs provide trustworthy optimization evidence.

## 2. Functional RTL Generation

Functional simulation is the primary evidence for normal RTL generation. Higher values are better.

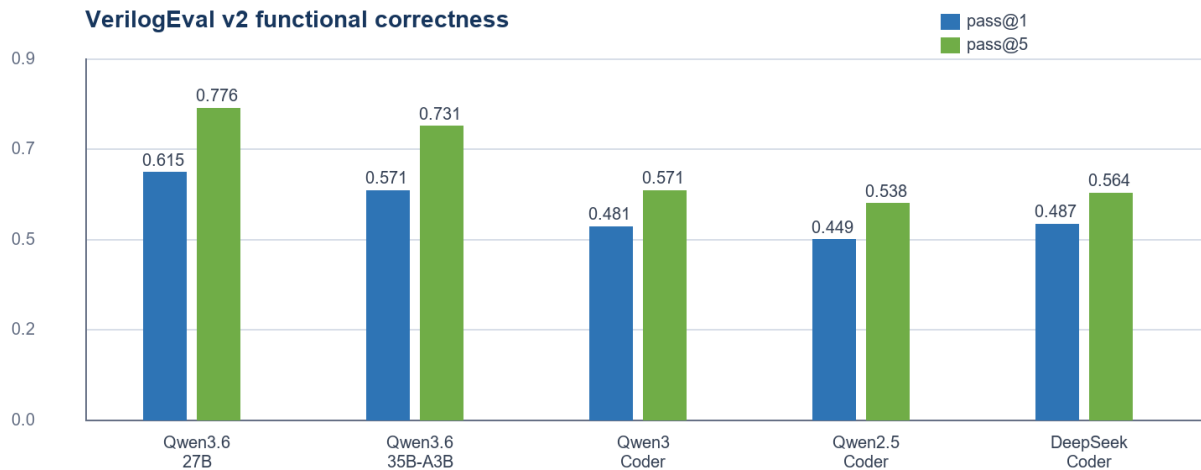


Figure 1. VerilogEval v2 functional pass rates under matched settings.

| Model           | VE pass@1 | VE pass@5 | RTLLM pass@1 |
|-----------------|-----------|-----------|--------------|
| Qwen3.6 27B     | 61.54%    | 77.56%    | 60.00%       |
| Qwen3.6 35B-A3B | 57.05%    | 73.08%    | 60.00%       |
| Qwen3 Coder     | 48.08%    | 57.05%    | 50.00%       |
| Qwen2.5 Coder   | 44.87%    | 53.85%    | 54.00%       |
| DeepSeek Coder  | 48.72%    | 56.41%    | 52.00%       |

### What this means

- 1 Qwen3.6 27B is the clearest general-purpose winner across both VerilogEval sampling conditions.
- 1 Qwen3.6 35B-A3B is consistently second on VerilogEval and ties the 27B model on RTLLM.
- 1 DeepSeek Coder is the best non-Qwen model on VerilogEval pass@1, but Qwen3 Coder slightly overtakes it on pass@5.
- 1 The pass@5 gain shows that sampling multiple candidates materially improves task recovery for every model.

### 3. Reliability and Failure Patterns

A model can produce syntactically valid RTL that is still behaviorally wrong. The benchmark therefore tracks extraction, compilation, and simulation failures separately.

| Model           | VE syntax pass@1 | VE functional pass@1 | Gap       |
|-----------------|------------------|----------------------|-----------|
| Qwen3.6 27B     | 83.97%           | 61.54%               | 22.43 pts |
| Qwen3.6 35B-A3B | 75.64%           | 57.05%               | 18.59 pts |
| Qwen3 Coder     | 86.54%           | 48.08%               | 38.46 pts |
| Qwen2.5 Coder   | 88.46%           | 44.87%               | 43.59 pts |
| DeepSeek Coder  | 85.90%           | 48.72%               | 37.18 pts |

**Key insight** The coder-specialized models are very good at producing parseable RTL, but their much lower simulation pass rates reveal semantic errors in timing, state transitions, reset behavior, or protocol logic.

#### VerilogEval pass@1 failure counts

| Model           | Passed | Simulation | Compile | Extraction |
|-----------------|--------|------------|---------|------------|
| Qwen3.6 27B     | 96     | 35         | 17      | 8          |
| Qwen3.6 35B-A3B | 89     | 29         | 26      | 12         |
| Qwen3 Coder     | 75     | 60         | 20      | 1          |
| Qwen2.5 Coder   | 70     | 68         | 18      | 0          |
| DeepSeek Coder  | 76     | 58         | 21      | 1          |

The dominant weakness for Qwen3 Coder, Qwen2.5 Coder, and DeepSeek Coder is simulation failure rather than extraction failure. Improving prompt packaging alone is unlikely to close this gap; the models need stronger functional reasoning or an explicitly separate repair loop.

## 4. Protocol Lint and RTL Optimization

### ProtocolLLM public lint

| Qwen3.6 27B | Qwen3.6 35B-A3B | Qwen3 Coder | Qwen2.5 Coder | DeepSeek Coder |
|-------------|-----------------|-------------|---------------|----------------|
| 77.78%      | 22.22%          | 77.78%      | 66.67%        | 66.67%         |

**Important limitation** ProtocolLLM public results are lint-only. They show whether output is structurally acceptable to Verilator, not whether the generated protocol implementation behaves correctly.

### RTL-OPT equivalence

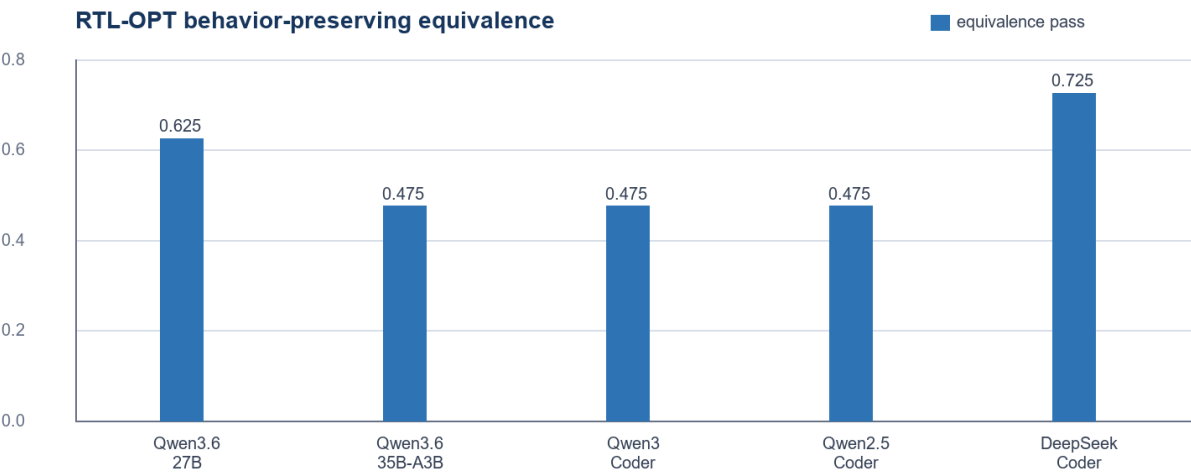


Figure 2. Fraction of RTL-OPT tasks that synthesized and remained equivalent to the original design.

| Model           | Equiv. pass | Reduced cells | Avg. cell ratio |
|-----------------|-------------|---------------|-----------------|
| Qwen3.6 27B     | 25 / 40     | 9 / 25        | 0.9124          |
| Qwen3.6 35B-A3B | 19 / 40     | 6 / 19        | 0.9003          |
| Qwen3 Coder     | 19 / 40     | 4 / 19        | 0.9472          |
| Qwen2.5 Coder   | 19 / 40     | 8 / 19        | 0.8709          |
| DeepSeek Coder  | 29 / 40     | 3 / 29        | 0.9739          |

### Interpretation

- 1 DeepSeek Coder preserves behavior most often, passing equivalence on 29 of 40 tasks.
- 1 Qwen2.5 Coder has the best average cell ratio among equivalent outputs, but it reaches equivalence on only 19 tasks.
- 1 Qwen3.6 27B offers the strongest balance: second-best equivalence coverage and the largest number of equivalence-passing reductions.
- 1 A lower cell ratio is only meaningful after equivalence passes; non-equivalent or synthesis-failing outputs are excluded from optimization claims.

## 5. Model Selection Guide

| Use case                             | Recommended model           | Why                                     | Caution                                 |
|--------------------------------------|-----------------------------|-----------------------------------------|-----------------------------------------|
| General RTL generation               | Qwen3.6 27B                 | Best functional pass@1 and pass@5       | Still fails roughly 38% of pass@1 tasks |
| Alternative high-performing baseline | Qwen3.6 35B-A3B             | Second on VerilogEval; ties RTLLM lead  | Lower syntax and extraction reliability |
| Behavior-preserving RTL rewrite      | DeepSeek Coder              | Best equivalence coverage               | Few equivalent outputs reduce cells     |
| Optimization candidate generation    | Qwen3.6 27B / Qwen2.5 Coder | More valid cell reductions / best ratio | Use equivalence as a hard gate          |
| Syntax-first prototyping             | Qwen2.5 Coder               | Best VerilogEval syntax rate            | Syntax does not imply correct behavior  |

### Recommended next experiments

- 1 Analyze simulation failures by semantic category: reset, cycle alignment, FSM transitions, width/sign errors, and handshake logic.
- 2 Add a separately reported repair-loop condition that feeds compiler or simulation errors back to the model.
- 3 Run ProtocolLLM with functional testbenches before making protocol-correctness claims.
- 4 Evaluate optimization quality with technology-mapped area, timing, and power after equivalence passes.
- 5 Repeat selected runs across seeds to quantify variance and confidence intervals.

### Operational achievement

The project now has a reusable, model-swappable benchmark workflow. Each run preserves raw responses, extracted RTL, logs, error logs, machine-readable results, summaries, and a per-run report in timestamped folders on Lanta. Committed manifests point to the authoritative output locations without placing large generated artifacts in Git.

## Appendix A. Exact Model IDs

| Report label    | Internal model ID               |
|-----------------|---------------------------------|
| Qwen3.6 27B     | qwen36-27b                      |
| Qwen3.6 35B-A3B | qwen36-35b-a3b                  |
| Qwen3 Coder     | qwen3-coder-30b-a3b-instruct    |
| Qwen2.5 Coder   | qwen25-coder-32b                |
| DeepSeek Coder  | deepseek-coder-v2-lite-instruct |

## Appendix B. Scope and Limitations

- 1 Results apply to the tested public benchmark versions, prompts, sampling settings, endpoints, and evaluator toolchain.
- 1 No prompt engineering or automated repair loop is included in the baseline condition.
- 1 Pass@5 measures whether at least one of five samples solves a task; it is not the same as single-sample reliability.
- 1 ProtocolLLM public evaluation is lint-only in the current harness.
- 1 RTL-OPT generic cell counts are an early proxy, not final PPA evidence from a technology-mapped flow.
- 1 One DeepSeek VerilogEval pass@5 attempt was incomplete and excluded; one ProtocolLLM attempt failed due to endpoint timeout and was rerun successfully.

## Appendix C. Reproducibility

The benchmark repository records model presets, deterministic benchmark configuration, timestamped output paths, failure categories, and comparison reports. Raw output folders remain on Lanta; Git tracks the configs, summary reports, and output manifests.